FACEATTEND · VIVA PREPARATION

Viva Questions & Answers

50 project questions your examiner is likely to ask, with short answers you can speak in the viva. They cover the project, workflow, AI models, anti-spoofing, security, database, testing and tricky questions.

1. Project basics
2. Workflow
3. AI models & transfer learning
4. Face matching
5. Anti-spoofing (liveness)
6. Security
7. Database
8. Features
9. Testing & limitations
10. Tricky questions
11. Numbers to remember
12. Memory tricks

How to answer: start with the one-line answer, then add one detail from your project (a model name, a number or a file). Short and confident beats long and unsure.

1 Project basics

Q1. What is your project?
A face recognition attendance system. A person stands in front of the camera and attendance is marked automatically, with checks against fake faces such as photos or screens.

Q2. Why did you choose this project?
Manual attendance is slow and allows proxy attendance. Face recognition is fast, contactless and hard to fake.

Q3. What technologies did you use?

- **Frontend:** React
- **Backend:** Node.js + Express
- **Database:** MongoDB (Mongoose)
- **Face AI:** @vladmandic/face-api on TensorFlow.js (WASM backend)
- **Anti-spoofing:** MiniFASNet models run with onnxruntime-node
- **Excel export:** ExcelJS

Q4. What are the main modules?
Admin console, kiosk (scanning), registration, recognition, liveness (anti-spoof), attendance, and audit log.

Q5. Who are the users of the system?
Admin: registers people, views and exports attendance, manages devices. **Students/staff:** scan their face at the kiosk.

Q6. What is a kiosk?
The attendance scanning screen with a camera, kept at the entrance, where people mark attendance by showing their face. It is the `/kiosk` page.

2 Workflow

Q7. Explain the full flow of your project.
Registration: admin captures 5 photos → each face becomes 128 numbers → the numbers are encrypted and saved.
Attendance: the kiosk camera sends frames → the system finds the face → checks quality → checks the person is real → asks for a head turn → matches the face → marks attendance.

Q8. Why do you take 5 photos at registration?
Different angles (straight, left, right, up, down) make recognition reliable when the person is not looking straight at the camera.

Q9. What happens if the same person is registered twice?
The system detects a duplicate face (distance below **0.42**) and warns the admin before saving.

Q10. Can attendance be marked twice in a day?
No. The database has a unique index on (student, day), so a second scan shows "Already marked today".

Q11. What does the kiosk show as a result?
"Attendance marked ✓", "Already marked today", or "Unable to verify identity" when it is not sure.

3 AI models & transfer learning

Q12. Which pre-trained models did you use?

Model	Job
SSD MobileNet v1	**Finds** the face in the picture
Face Landmark 68	**Marks 68 points** (eyes, nose, mouth, jaw)
Face Recognition ResNet	Turns the face into **128 numbers**
MiniFASNetV2	**Anti-spoof:** real face or fake?
MiniFASNetV1SE	**Second anti-spoof** check (wider view)

Q13. What is transfer learning?
Reusing a model that was already trained on a huge dataset for a new task, instead of training from zero. It is like someone who can ride a bicycle learning a motorbike faster.

Q14. Did you use transfer learning? Which type?
Yes, as **feature extraction**: the pre-trained models stay frozen, and only their output (the 128 numbers and scores) is used. No fine-tuning is done.

Q15. Did you train any model?
No. Registration only stores the face numbers; it does not train or change any model.
Remember: "Registration stores; it doesn't train."

Q16. Where are the models used in your code?
`faceEngine.js` loads and runs the detection, landmark and recognition models. `liveness.js` runs the two anti-spoof models.

Q17. What is the 68-landmark model used for?
Head angle (yaw/roll), face alignment, blink detection and the head-turn challenge.

Q18. What input sizes do the models use?
The recognition model takes 150×150 pixels. The anti-spoof models take 80×80 pixels.

4 Face matching

Q19. What is a face embedding (descriptor)?
A list of 128 numbers that describes a face. Photos of the same person give very similar numbers.

Q20. How do you compare two faces?
With the **Euclidean distance** between their 128-number lists. A small distance means the same person, and **≤ 0.5** counts as a match.

Q21. How do you avoid a wrong person being marked (false positive)?

- The distance must be ≤ **0.5**.
- The best match must beat the second-best person by at least **0.06** (margin).
- The same result must hold for **3 frames** in a row, and more frames when the match is weaker.
- If unsure, the kiosk shows "Unable to verify identity" instead of guessing.

Q22. Why did you choose 0.5 as the threshold?

It is the standard value for this model. It can be calibrated on our own data with the `npm run evaluate` script.

5 Anti-spoofing (liveness)

Q23. What is liveness detection?

Checking that a real, live person is in front of the camera, not a photo, video or phone screen.

Q24. How did you implement liveness?

1. Passive: two MiniFASNet models give a "real" score, which must be ≥ **0.6**.
2. Active: a challenge where the person must slightly turn their head left or right within 10 seconds.

Q25. Why two anti-spoof models?

One looks closely at the face (2.7× crop) and the other looks at a wider area including the background (4.0× crop). Together they are more accurate.

Q26. Can someone fool the system with a photo?

A printed photo or screen usually fails the passive check, and a still photo cannot turn its head for the challenge. Failed attempts are recorded in the audit log.

6 Security

Q27. How do you protect face data?

Face numbers are encrypted with **AES-256-GCM** in the database, and no face photos are stored.

Q28. How is the admin password stored?

Hashed with **bcrypt** (cost 12), never stored as plain text.

Q29. How does login work?

A **JWT** token is kept in an httpOnly, SameSite=Strict cookie. After **5 wrong passwords** the account locks for 15 minutes.

Q30. Which attacks did you protect against?

- **CSRF:** custom header plus origin check
- **XSS:** Content Security Policy headers (helmet)
- **NoSQL injection:** input validation with zod
- **Brute force:** rate limits and account lockout
- **Fake uploads:** image file type and size checks

Q31. Can anyone open the kiosk and use it?
No. The admin must activate the device first, and each device gets its own login, which the admin can deactivate.

Q32. Where are secret keys kept?
In the `.env` file, which is never committed to Git.

7 Database

Q33. Which database and why?
MongoDB. It is flexible for storing documents such as students with several face samples, and it works naturally with Node.js.

Q34. What collections are there?
Students, Attendance, Admin, Device and AuditLog.

Q35. What is stored in an attendance record?
Name, roll number, date, check-in time, status (Present/Late), and verification details (distance, confidence, liveness score).

Q36. What is the audit log?
A record of important actions, such as logins, registrations, edits, exports and failed spoof attempts, with who did them and when.

8 Features

Q37. How is the attendance report generated?
The admin clicks **Export Excel**, and the backend creates an `.xlsx` file with ExcelJS, filtered by date, name or status.

Q38. What is "Late" status?
A check-in after the set time (`LATE_AFTER` , for example 09:30) is marked Late, using the Asia/Kolkata time zone.

Q39. What quality checks do you do before using a face?
Face size, sharpness (blur), brightness, and head angle. A bad frame gets a hint such as "Move closer to the camera" or "Improve lighting".

Q40. What does the dashboard show?
Active users, present / late / absent today, today's check-ins, the last 7 days, and warnings for users who need re-registration or possible duplicate faces.

9 Testing & limitations

Q41. How did you test the project?
58 backend tests (recognition, security, attendance), **9 frontend tests**, and end-to-end tests in a real browser with a fake camera.

Q42. What is the accuracy?

The ResNet recognition model is about 99% accurate on the LFW benchmark. Real accuracy depends on the thresholds, which can be tuned with the evaluate scripts.

Q43. What are the limitations?

Very poor lighting, identical twins, and high-quality 3D masks can cause problems, and a camera is required.

Q44. What is the future scope?

Mobile app, cloud deployment with HTTPS, SMS/email alerts, multiple classes or departments, and depth-camera liveness.

10 Tricky questions

Q45. Why didn't you train your own CNN?

Training needs millions of face images and powerful GPUs. Pre-trained models already give high accuracy, which is exactly the benefit of transfer learning.

Q46. Is this classification? Why use distance?

No, it is comparison. Each face is turned into numbers and compared by distance, so new people can be added without retraining.

Q47. What happens when a new student joins?

The admin registers them with 5 photos. No retraining is needed.

Q48. Where does the face processing happen — browser or server?

On the **server**. The browser only sends camera frames, so the models and the face data are not exposed to the user.

Q49. Why "Unable to verify identity" instead of the closest match?

Marking the wrong person is worse than asking someone to try again. Avoiding false positives is the system's top priority.

Q50. What if the encryption key is lost?

The stored face data cannot be decrypted, so everyone must register again. That is why the key must be backed up safely.

11 Numbers to remember

What	Value
Pre-trained models	**5**
Face embedding size	**128** numbers
Landmark points	**68**
Photos at registration	**5** (straight, left, right, up, down)
Match threshold (distance)	**≤ 0.5**
Minimum margin to second-best	**0.06**
Duplicate face threshold	**0.42**
Stable frames needed	**3**
Liveness "real" score	**≥ 0.6**
Head-turn challenge time	**10** seconds
Recognition input / anti-spoof input	**150×150** / **80×80** pixels
Encryption	**AES-256-GCM**
Password hashing	**bcrypt**, cost 12
Account lock	**5** wrong passwords → **15** minutes
Backend / frontend tests	**58** / **9**
Ports	Backend **5000**, frontend **3000**, MongoDB **27017**

12 Memory tricks

Find → Point → Guard → Fingerprint
The models in order: detection, landmarks, anti-spoof, recognition.

Freeze → Extract → Compare
How transfer learning works here: frozen models, 128 numbers, distance.

Stores, doesn't train
Registration saves encrypted face numbers; no model is trained.

60-second answer: "My project is a face recognition attendance system built with React, Node.js and MongoDB. It uses five pre-trained models through transfer learning, as feature extraction: SSD MobileNet finds the face, Face Landmark 68 marks points, MiniFASNet checks the face is real, and ResNet turns it into 128 numbers. These numbers are compared by Euclidean distance with a 0.5 threshold, a margin check and 3 stable frames, so a wrong person is never marked. Face data is encrypted with AES-256, and the admin can export attendance to Excel."